

Deep Learning Fall 2025

Prof. Gustavo Sandoval

Due Date: **5pm Friday October 3, 2025**

- You are encouraged to discuss ideas with each other. But you **must acknowledge** any collaborators, including students, ChatGPT, books, online sources, etc. and you **must** write up your own solutions individually.
- We **require** answers to theory questions to be written in L^AT_EX or markdown. Figures can be drawn by hand, but **must** be scanned and inserted into your document. We will not accept hand-written answers.
- We **require** code for programming questions to be submitted as a Jupyter notebook. It is important to include sufficient documentation so that the grader can understand your code. Use the text cells in Jupyter notebook to provide explanations.
- Upload your .pdf, .ipynb and any other files to **Gradescope** in a single PDF.

Problem 1: Warmup: Fun with vector calculus. (15 pts)

- (a) Suppose that z is a vector with n elements. We would like to compute the gradient of the following function with respect to z :

$$y = \text{softmax}(z) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

show that the gradient of y with respect to z is given by:

$$\frac{\partial y}{\partial z} = y \cdot (I - y)$$

Where I is the identity matrix.

- (b) Suppose that A is a $d \times d$ matrix and b is a d -dimensional vector. Write down the gradient of the following function with respect to A and b :

$$f(x) = \frac{1}{2} \|Ax - b\|_2^2$$

- (c) What type of problems are better solved using the softmax activation function over the sigmoid activation function? Explain your answer.

Problem 2: Linear Regression with non-standard losses (15 pts)

The Huber loss is a loss function used in robust regression, that is less sensitive to outliers in data than the squared error loss we saw in class. The Huber loss is defined as:

$$L_\delta(w) = \sum_{i=1}^n \begin{cases} \frac{1}{2}(y_i - w^T x_i)^2 & \text{if } |y_i - w^T x_i| \leq \delta \\ \delta |y_i - w^T x_i| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

where w is the weight vector, x_i is the i -th data point, y_i is the i -th target value, and δ is a hyperparameter.

- (a) Write down the gradient of the Huber loss with respect to w .
- (b) Can you write down the optimal linear model in closed form for the Huber loss function? Explain any difficulties and how they compare to the standard least squares problem.
- (c) Add an l_2 -regularization term to the Huber loss function. Write down the new objective function and discuss the effect of this regularization on the solution, especially in the presence of outliers.
- (d) In about 2-3 sentences, relate how you solved this problem to the 3-step recipe for machine learning we discussed in class, and reflect on how the Huber loss improves over the least squares loss in terms of robustness to outliers.

Problem 3: Adam vs. SGD with Momentum. (15 pts)

Recall the Adam update rules for a parameter w at iteration t :

$$g_t = \nabla \ell_t(w_t), \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

with bias corrections

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

and update rule

$$w_{t+1} = w_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}.$$

1. Write the update explicitly for the scalar case (a single parameter w).
2. Compare this update to SGD with momentum:

$$v_t = \mu v_{t-1} + \alpha g_t, \quad w_{t+1} = w_t - v_t.$$

3. In 3–4 sentences, explain how Adam differs from momentum-based SGD in terms of (i) learning-rate adaptation and (ii) treatment of noisy or infrequent features.

Problem 4: Improving the FashionMNIST classifier (15 pts)

In the first demo, we classified the MNIST dataset with a logistic regression architecture. Now, we'll use a neural network to improve the performance.

Repeat the same experiment in the demo but with the FashionMNIST dataset. You can load the dataset using the following pytorch code:

```
trainingdata = torchvision.datasets.FashionMNIST('./FashionMNIST/',
        train=True, download=True, transform=torchvision.transforms.ToTensor())

testdata = torchvision.datasets.FashionMNIST('./FashionMNIST/',
        train=False, download=True, transform=torchvision.transforms.ToTensor())
```

Now use a (dense) neural network with 2 hidden layers. The first layer should have 128 neurons, the second should have 64 all with ReLU activations. Remember to then use one last layer so the output is 10-dimensional (for the 10 classes). Display train- and test-loss curves. You may have to tweak the total number of training epochs to get reasonable performance.

Finally, sample any three images from the validation set and visualize the predicted class probabilities for each sample. Comment on what you observe from the plots.

Problem 5: Optimizer comparison on FashionMNIST. (15 pts)

In Problem 4, you trained a 2-hidden-layer MLP (128–64 ReLU, 10 outputs) on Fashion-MNIST using a basic optimizer. Now repeat the experiment with two different optimizers:

1. SGD with momentum ($\mu = 0.9$).
2. Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$).

Keep the architecture, initialization, and batch size the same as before. Run both optimizers for the same number of epochs (or updates) and tune the learning rate for each using a small grid (e.g. $\{10^{-3}, 3 \times 10^{-3}, 10^{-2}\}$).

Tasks:

- Plot the training loss and validation accuracy curves for each optimizer.
- Report the final test accuracy.
- In 3–4 sentences, discuss which optimizer converges faster and which one generalizes better in this setting.

Problem 6: Implementing Backpropagation (25 pts)

Open the (incomplete) Jupyter notebook provided as an attachment to this homework in Google Colab (or other Python IDE of your choice) and complete the missing items.

In the third demo, we worked with autodiff. Autodiff enables us to implicitly store how to calculate the gradient when we call backward. We implemented some basic operations (addition, multiplication, power, and ReLU).

In this homework problem, you will implement backprop for more complicated operations directly. Instead of using autodiff, you will manually compute the gradient of the loss function for each parameter.